

# Spring Security Internals

Deep Dive into Authentication, Authorisation, and Filter Chains

Chapter 08 — Spring Deep Dive Series

|                                                |                                                 |
|------------------------------------------------|-------------------------------------------------|
| <b>A. Authentication vs Authorisation</b>      | Who are you? vs What can you do?                |
| <b>B. SecurityFilterChain</b>                  | Every request passes through filters            |
| <b>C. Authentication Flow</b>                  | UsernamePasswordAuthenticationFilter internals  |
| <b>D. UserDetailsService &amp; UserDetails</b> | Loading users from database                     |
| <b>E. PasswordEncoder</b>                      | BCrypt, cost factor, timing-safe matches()      |
| <b>F. SecurityContext &amp; Holder</b>         | ThreadLocal storage and async contexts          |
| <b>G. HTTP Security Config</b>                 | permitAll, authenticated, hasRole, hasAuthority |
| <b>H. JWT Authentication</b>                   | Stateless auth, token validation filter         |
| <b>I. Method Security</b>                      | @PreAuthorize, @PostAuthorize, AOP internals    |
| <b>J. CSRF Protection</b>                      | Synchroniser token pattern explained            |

# A Authentication vs Authorisation

The fundamental difference every developer must know

## 1. Plain English Explanation

Authentication answers: **Who are you?** It is the process of verifying the identity of a principal — typically by checking a credential (password, token, certificate) against a stored record. Until authentication succeeds, Spring Security treats the request as anonymous.

Authorisation answers: **What are you allowed to do?** Once Spring knows who the user is, authorisation decides whether that authenticated principal has permission to access a resource or perform an action. These are two completely separate concerns, executed in that strict order.

## 2. Real-World Analogy

Think of a corporate office building. At the entrance, a security guard checks your employee badge and confirms you are really Alice (authentication). Once inside, your badge's access level determines which floors and rooms you can enter — the server room might require a senior-engineer badge (authorisation). Knowing who you are does not automatically grant you access everywhere.

## 3. Minimal Code Example

```
@RestController
public class DemoController {

    // Authentication enforced by Spring Security filter chain
    // Authorisation enforced by the security config rule below
    @GetMapping("/admin/report")
    public String adminReport() {
        return "Sensitive report";
    }

    @GetMapping("/profile")
    public String profile() {
        return "My profile";
    }
}
```

```
@Bean
public SecurityFilterChain chain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth
        .requestMatchers("/admin/**").hasRole("ADMIN") // authorisation
        .anyRequest().authenticated() // authentication required
    ).formLogin(Customizer.withDefaults());
    return http.build();
}
```

## 4. Diagram

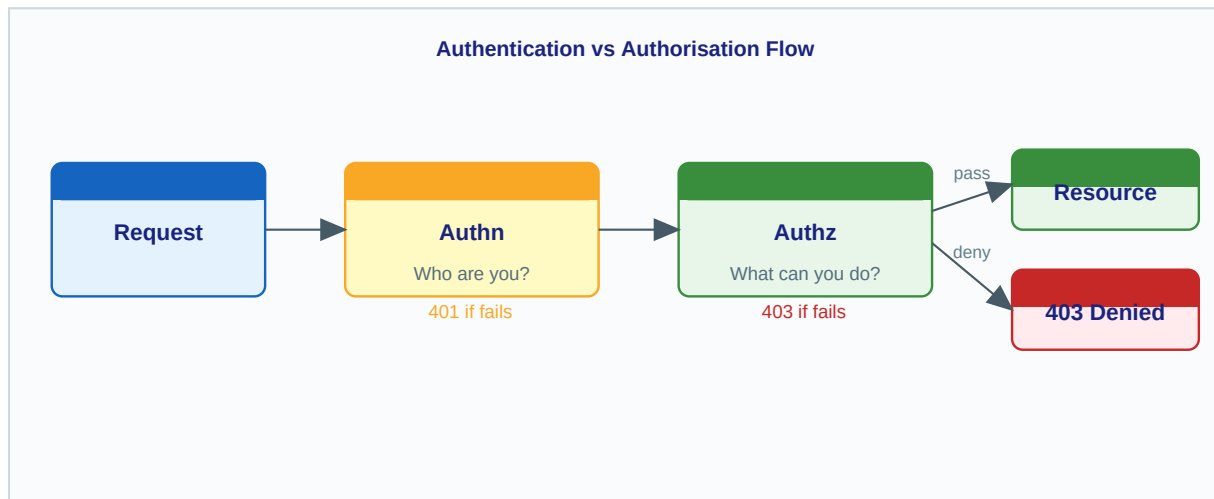


Figure A — Authentication precedes Authorisation; each has its own failure code

## 5. Behind the Scenes

Spring Security models the authenticated user as an **Authentication** object (interface in `org.springframework.security.core.Authentication`). Before authentication, the `SecurityContext` holds an anonymous token. After successful login, it holds a **UsernamePasswordAuthenticationToken** with `isAuthenticated() = true`.

**Authorisation** is handled by **AuthorizationFilter** (since Spring Security 5.5, replacing `FilterSecurityInterceptor`). It calls **AuthorizationManager.check()** which inspects the `Authentication`'s `GrantedAuthority` list against the configured rules.

Key classes: **SecurityContextHolder**, **Authentication**, **GrantedAuthority**, **AuthorizationManager**, **AuthorizationFilter**.

## 6. Realistic Practical Example

```
// Checking current user's identity and roles in service code
@Service
public class OrderService {
    public Order placeOrder(OrderRequest req) {
        Authentication auth =
            SecurityContextHolder.getContext().getAuthentication();
        String username = auth.getName(); // authentication info
        boolean isAdmin = auth.getAuthorities()
            .stream()
            .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));
        if (!isAdmin && req.getTotalValue() > 10_000) {
            throw new AccessDeniedException("Large orders need admin");
        }
        return orderRepo.save(new Order(username, req));
    }
}
```

## 7. Common Mistakes and Misconceptions

- [WARN] Confusing 401 (Unauthenticated) with 403 (Unauthorised) — they mean different things
- [WARN] Trying to authorise before authenticating — you cannot check roles if identity is unknown
- [WARN] Putting business-logic access checks only in the controller layer instead of the service
- [OK] Always authenticate first, then authorise; Spring Security enforces this order
- [OK] Use 401 for missing/invalid credentials and 403 for insufficient permissions

## B SecurityFilterChain

How every HTTP request passes through an ordered chain of filters

### 1. Plain English Explanation

A **SecurityFilterChain** is an ordered list of servlet filters that Spring Security applies to incoming HTTP requests. Each filter has a single responsibility: one logs requests, one handles CORS, one processes authentication, one enforces authorisation, etc. A request travels down the chain; any filter can halt it and return a response early.

Spring allows multiple SecurityFilterChain beans — each matches a URL pattern — so you can have different security rules for `/api/**` (stateless JWT) and `/admin/**` (form login).

### 2. Real-World Analogy

Imagine airport security: first you queue at check-in (session management filter), then passport control (authentication filter), then baggage X-ray (CSRF filter), then gate boarding pass check (authorisation filter). Each checkpoint is independent, runs in order, and any one can turn you away. You only reach the gate (your controller) if all pass.

### 3. Minimal Code Example

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/**")           // only matches /api/**
            .csrf(csrf -> csrf.disable())         // stateless API
            .sessionManagement(s ->
                s.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated());
        return http.build();
    }
}
```

### 4. Diagram

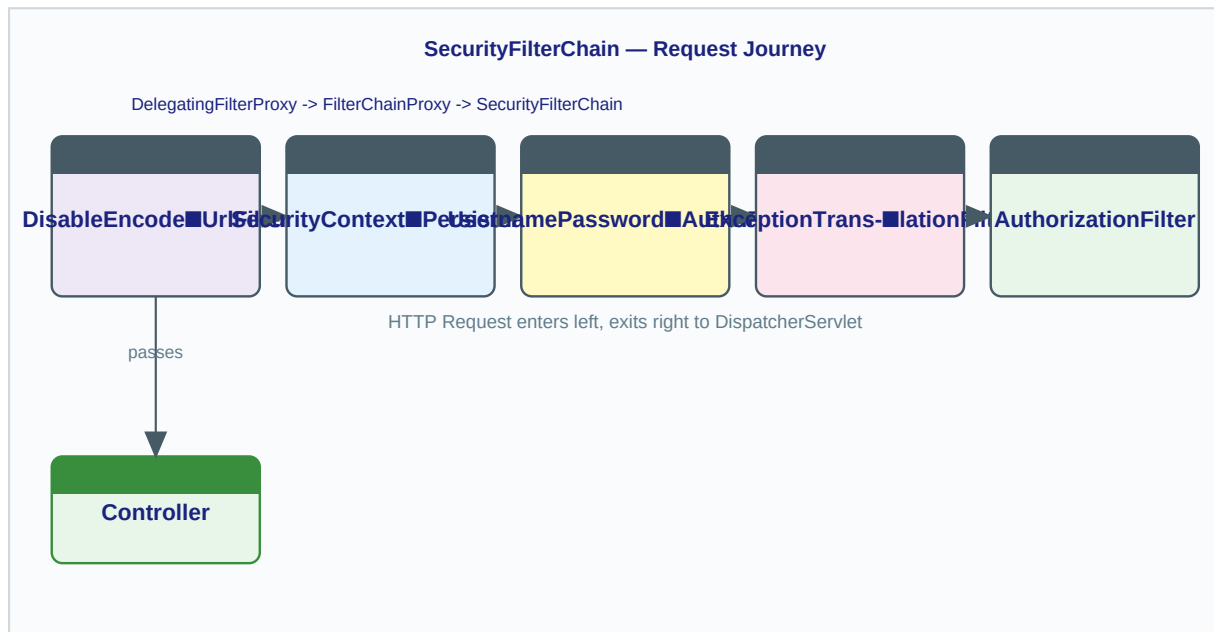


Figure B — Filters execute left to right; AuthorizationFilter is last

## 5. Behind the Scenes

**DelegatingFilterProxy** is a plain `javax.servlet.Filter` registered with the servlet container. It lazily looks up a Spring bean named 'springSecurityFilterChain' from the `ApplicationContext` and delegates all filter calls to it.

**FilterChainProxy** is that Spring bean. It holds a list of `SecurityFilterChain` beans and, for each request, iterates them in order, picking the first chain whose `RequestMatcher` matches the request URL. It then invokes that chain's filters sequentially via a `VirtualFilterChain`.

Standard filter order (from `FilterOrderRegistration`, Spring Security 6.x):

- 100 — `DisableEncodeUrlFilter`
- 200 — `ForceEagerSessionCreationFilter`
- 300 — `ChannelProcessingFilter`
- 400 — `WebAsyncManagerIntegrationFilter`
- 500 — `SecurityContextHolderFilter` (replaces `SecurityContextPersistenceFilter`)
- 700 — `CsrfFilter`
- 800 — `LogoutFilter`
- 1100 — `UsernamePasswordAuthenticationFilter`
- 1500 — `DefaultLoginPageGeneratingFilter`
- 3000 — `ExceptionTranslationFilter`
- 3100 — `AuthorizationFilter`

## 6. Realistic Practical Example

```
@Bean
@Order(1)
public SecurityFilterChain actuatorChain(HttpSecurity http) throws Exception {
    http.securityMatcher("/actuator/**")
        .authorizeHttpRequests(a -> a.anyRequest().hasRole("OPS"))
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

```
@Bean
@Order(2)
public SecurityFilterChain defaultChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(a -> a
        .requestMatchers("/public/**").permitAll()
        .anyRequest().authenticated())
        .formLogin(Customizer.withDefaults());
    return http.build();
}
```

## 7. Common Mistakes and Misconceptions

**[WARN] Forgetting @Order — without it, both chains match every request (first-registered wins)**

**[WARN] Disabling CSRF globally instead of only for stateless API chains**

**[WARN] Adding a custom filter with addFilterBefore without knowing the reference filter's order**

**[OK] Use securityMatcher() to scope each chain to specific URL patterns**

**[OK] Always inspect the filter chain in debug logs: logging.level.org.springframework.security=DEBUG**

## C

### Authentication Flow

What happens step-by-step when you POST /login

## 1. Plain English Explanation

When a user submits a login form, Spring Security's **UsernamePasswordAuthenticationFilter** intercepts the POST request, extracts the username and password, and orchestrates the entire authentication process. If successful, the authenticated user is stored in the SecurityContext and the user is redirected to the originally requested page.

## 2. Real-World Analogy

Think of a hotel check-in desk. You hand over your name and ID (username + password). The receptionist (filter) passes your details to the back-office system (AuthenticationManager). The system checks the guest list (UserDetailsService) and verifies your ID matches (PasswordEncoder). If everything checks out, you get your room key (Authentication token stored in SecurityContext). If not, you are turned away (AuthenticationFailureHandler).

## 3. Minimal Code Example

```
// Spring Boot auto-configures this when spring-boot-starter-security is on classpath
// POST /login with form params username= and password=
// UsernamePasswordAuthenticationFilter handles it automatically

@Bean
public SecurityFilterChain chain(HttpSecurity http) throws Exception {
    http
        .formLogin(form -> form
            .loginPage("/login")           // custom login page
            .defaultSuccessUrl("/dashboard")
            .failureUrl("/login?error=true")
        )
        .authorizeHttpRequests(a -> a
            .requestMatchers("/login").permitAll()
            .anyRequest().authenticated());
    return http.build();
}
```

## 4. Diagram

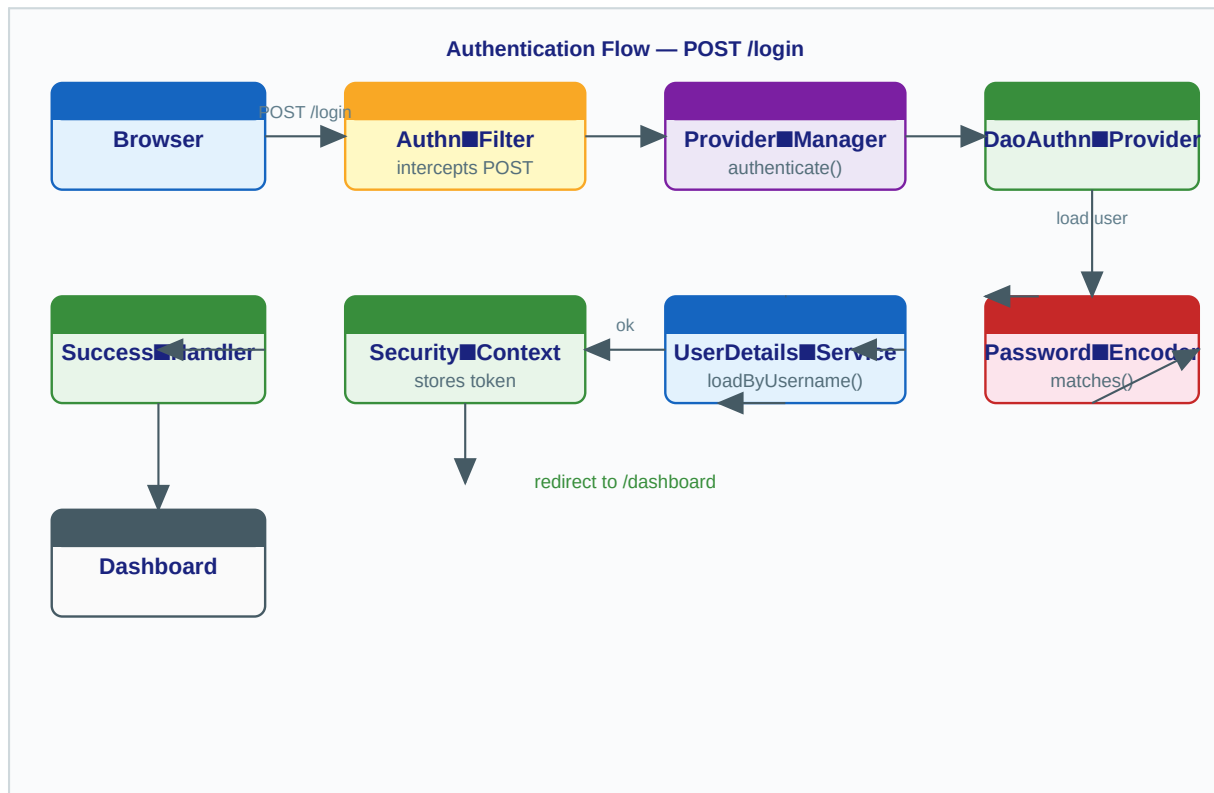


Figure C — Complete POST /login flow through ProviderManager to SecurityContext

## 5. Behind the Scenes

**UsernamePasswordAuthenticationFilter** extends `AbstractAuthenticationProcessingFilter`. It intercepts POST /login, calls `attemptAuthentication()` which creates an **UsernamePasswordAuthenticationToken** (unauthenticated) and passes it to `AuthenticationManager`.

**ProviderManager** implements `AuthenticationManager`. It iterates its list of **AuthenticationProvider** beans. For form login, **DaoAuthenticationProvider** is selected because it supports `UsernamePasswordAuthenticationToken`.

**DaoAuthenticationProvider** calls `UserDetailsService.loadUserByUsername(username)` to retrieve the stored user. It then calls `PasswordEncoder.matches(rawPassword, storedHash)`. If successful, it returns a fully authenticated `UsernamePasswordAuthenticationToken`.

Back in the filter, `successfulAuthentication()` is called: the token is stored in `SecurityContextHolder`, `HttpSessionSecurityContextRepository` saves the context to the session, and `SavedRequestAwareAuthenticationSuccessHandler` redirects to the original URL.

## 6. Realistic Practical Example

```

@Component
public class CustomAuthSuccessHandler
    implements AuthenticationSuccessHandler {

    @Override
    public void onAuthenticationSuccess(
        HttpServletRequest req,
        HttpServletResponse res,
        Authentication auth) throws IOException {
        String role = auth.getAuthorities().iterator()
            .next().getAuthority();
        if (role.equals("ROLE_ADMIN")) {

```



```
        res.sendRedirect("/admin/dashboard");
    } else {
        res.sendRedirect("/user/home");
    }
}
```

## 7. Common Mistakes and Misconceptions

[WARN] Expecting POST /login to work without permitAll() on that URL — the filter runs before auth

[WARN] Overriding loadUserByUsername to return null instead of throwing UsernameNotFoundException

[WARN] Storing the raw password in UserDetails.getPassword() instead of the encoded hash

[OK] Always throw UsernameNotFoundException from loadUserByUsername — never return null

[OK] The filter's URL defaults to POST /login — change it with loginProcessingUrl() if needed

## D

# UserDetailsService and UserDetails

How Spring loads and represents a user from the database

## 1. Plain English Explanation

**UserDetailsService** is a single-method interface: `loadUserByUsername(String username)`. Its job is to retrieve user data from wherever you store it (database, LDAP, in-memory map) and return it as a **UserDetails** object. Spring Security does not care how you fetch the data — it only cares that you return a valid **UserDetails** or throw `UsernameNotFoundException`.

**UserDetails** is the contract for what a user object must expose: username, password (encoded), granted authorities, and four boolean account-status flags.

## 2. Real-World Analogy

**UserDetailsService** is like an HR department. When security needs to verify an employee (`loadUserByUsername`), HR goes to the personnel file cabinet (database) and retrieves the employee record (**UserDetails**). The record contains their name, role (`ROLE_ENGINEER`), and whether their employment is still active (`isEnabled`, `isAccountNonExpired`, etc.).

## 3. Minimal Code Example

```
@Service
public class MyUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository repo;

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {
        User user = repo.findByUsername(username)
            .orElseThrow(() ->
                new UsernameNotFoundException("Not found: " + username));
        return org.springframework.security.core.userdetails.User
            .withUsername(user.getUsername())
            .password(user.getPasswordHash()) // BCrypt hash from DB
            .roles(user.getRole())           // e.g. "ADMIN" -> ROLE_ADMIN
            .accountLocked(!user.isActive())
            .build();
    }
}
```

## 4. Diagram

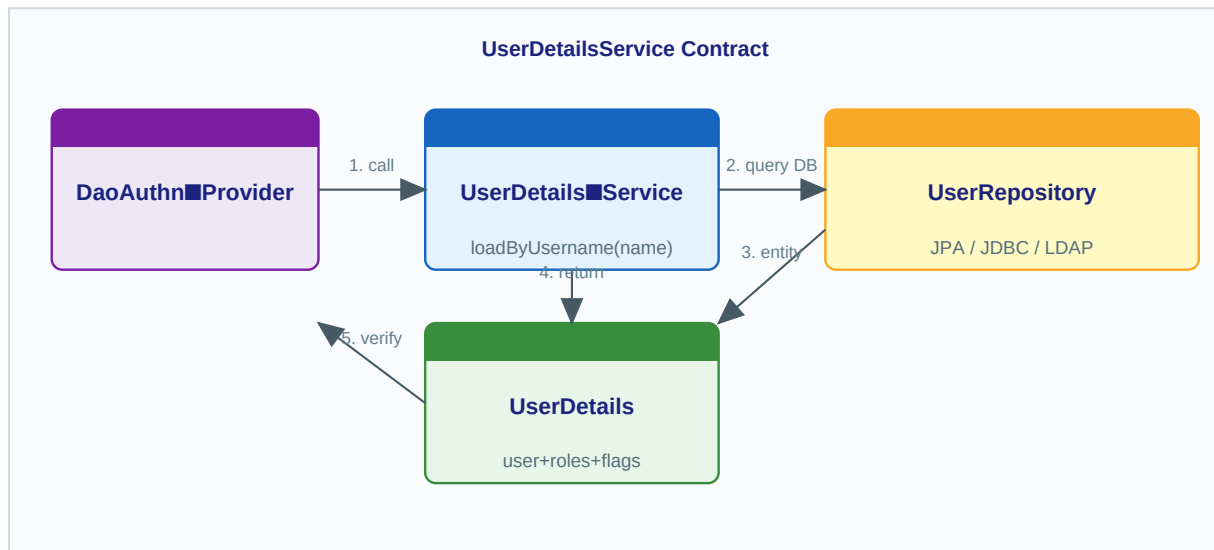


Figure D — DaoAuthenticationProvider uses UserDetailsService to load and verify the user

## 5. Behind the Scenes

`DaoAuthenticationProvider.retrieveUser()` calls `userDetailsService.loadUserByUsername()`. It wraps any `UsernameNotFoundException` in an `InternalAuthenticationServiceException`. After retrieval it calls `preAuthenticationChecks.check(userDetails)` which verifies `isAccountNonLocked`, `isEnabled`, `isAccountNonExpired`.

It then calls `additionalAuthenticationChecks()` which invokes `passwordEncoder.matches(presentedPassword, userDetails.getPassword())`. Only after both checks pass does it call `postAuthenticationChecks.check()` to verify `isCredentialsNonExpired`.

The returned `UserDetails` is wrapped inside a fully-authenticated `UsernamePasswordAuthenticationToken` containing the `UserDetails` as the principal and the `GrantedAuthority` list from `UserDetails.getAuthorities()`.

## 6. Realistic Practical Example

```

@Entity
public class AppUser {
    @Id @GeneratedValue
    private Long id;
    private String username;
    private String passwordHash;
    @ElementCollection(fetch = FetchType.EAGER)
    private Set roleNames;
    private boolean enabled;
}

@Service
public class AppUserDetailsService implements UserDetailsService {
    @Autowired AppUserRepository repo;

    @Override
    public UserDetails loadUserByUsername(String u) {
        AppUser appUser = repo.findByUsername(u)
            .orElseThrow(() -> new UsernameNotFoundException(u));
        List authorities = appUser.getRoleNames().stream()
            .map(SimpleGrantedAuthority::new).toList();
        return new org.springframework.security.core.userdetails
            .User(appUser.getUsername(), appUser.getPasswordHash(),
                appUser.isEnabled(), true, true, true, authorities);
    }
}
  
```

```
}  
}
```

## 7. Common Mistakes and Misconceptions

[WARN] Using FetchType.LAZY on roles — causes LazyInitializationException outside transaction

[WARN] Returning null from loadUserByUsername — Spring treats null as no user found, but incorrectly

[WARN] Storing raw passwords in the DB — always store BCrypt hashes

[OK] Always use FetchType.EAGER (or a JOIN FETCH query) for the roles collection

[OK] Throw UsernameNotFoundException — DaoAuthenticationProvider catches and wraps it correctly

## E

## PasswordEncoder

BCrypt, cost factor, and timing-safe password verification

### 1. Plain English Explanation

**PasswordEncoder** is a Spring Security interface with two key methods: `encode(rawPassword)` and `matches(rawPassword, encodedPassword)`. The default and recommended implementation is **BCryptPasswordEncoder**. It is a one-way adaptive hash function: you can never reverse it to get the original password. Every call to `encode()` produces a different hash even for the same input because it generates a random salt each time.

The **cost factor** (also called work factor or strength) controls how long hashing takes. BCrypt default is 10, meaning  $2^{10} = 1024$  rounds of the Blowfish cipher. Doubling the cost factor doubles the compute time, making brute-force attacks exponentially harder.

### 2. Real-World Analogy

BCrypt is like a combination safe where opening it requires turning the dial 1024 times in the right sequence. You cannot shortcut those turns. Each safe (hash) also has a unique serial number (salt) embedded in the lock, so even two identical passwords produce completely different safes. An attacker must crack each safe individually.

### 3. Minimal Code Example

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12); // cost factor 12
}

// Encoding at registration
String raw = "mySecret123";
String hash = encoder.encode(raw);
// e.g.: $2a$12$abcdefgghijklmnopqrstuSomeHashValueHere...

// Matching at login - timing-safe comparison
boolean ok = encoder.matches(raw, hash); // true
boolean bad = encoder.matches("wrong", hash); // false
// Both calls take roughly the same time regardless of match result
```

### 4. Diagram

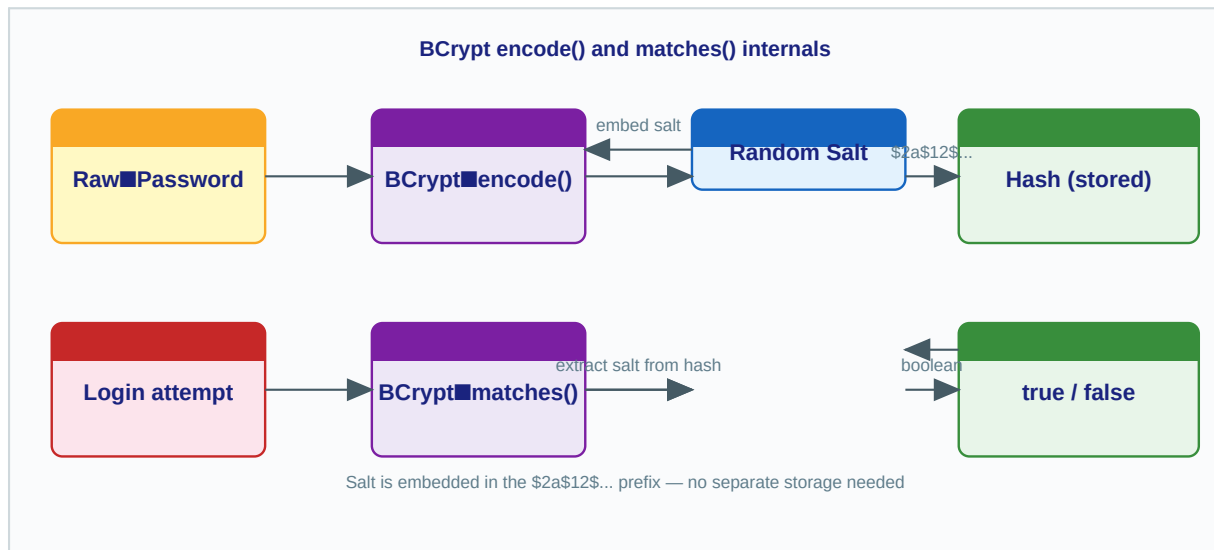


Figure E — BCrypt always embeds the salt in the stored hash string

## 5. Behind the Scenes

`BCryptPasswordEncoder` delegates to the `BCrypt` library. The stored hash has format: `$2a$[cost]$[22-char-salt][31-char-hash]`. The salt is base64-encoded, 128 bits of `SecureRandom`.

**`matches(raw, encoded)`** extracts the version, cost, and salt from the encoded string, re-hashes the raw password with that exact salt and cost, then does a byte-array comparison. Because the same salt produces the same hash, the comparison is valid. It uses **`MessageDigest.isEqual()`** — a constant-time comparison — to prevent timing attacks that could reveal partial matches.

**`DelegatingPasswordEncoder`** is the preferred factory approach: it prefixes the encoded string with `{bcrypt}`, `{pbkdf2}`, `{scrypt}`, etc., allowing multiple encoders and on-the-fly migration between algorithms without rehashing all passwords at once.

## 6. Realistic Practical Example

```

@Service
public class UserRegistrationService {

    @Autowired PasswordEncoder encoder;
    @Autowired UserRepository repo;

    public void register(String username, String rawPassword) {
        if (repo.existsByUsername(username)) {
            throw new IllegalArgumentException("Username taken");
        }
        String hash = encoder.encode(rawPassword); // BCrypt hash
        repo.save(new User(username, hash, "ROLE_USER"));
    }

    public boolean upgradePasswordIfNeeded(User u, String raw) {
        if (encoder.upgradeEncoding(u.getPasswordHash())) {
            u.setPasswordHash(encoder.encode(raw));
            repo.save(u);
            return true;
        }
        return false;
    }
}

```

## 7. Common Mistakes and Misconceptions

[WARN] Using cost factor 4 in production for speed — it makes brute-force trivially fast

[WARN] Storing the salt separately — BCrypt embeds it; storing it separately doubles exposure

[WARN] Using SHA-256 directly — it is fast by design and vulnerable to GPU rainbow tables

[OK] Use cost factor 10-12 in production; use cost factor 4 only in unit tests for speed

[OK] Use `PasswordEncoderFactories.createDelegatingPasswordEncoder()` for new apps

## 1. Plain English Explanation

**SecurityContext** is a simple container that holds the currently authenticated **Authentication** object. **SecurityContextHolder** is the global access point to the SecurityContext. By default it stores the context in a **ThreadLocal** — one context per thread — so every piece of code running in a request thread can call `SecurityContextHolder.getContext().getAuthentication()` and get the right user without passing the Authentication object around manually.

## 2. Real-World Analogy

Think of a restaurant where each waiter (thread) carries a notepad (ThreadLocal). The notepad records which table they are currently serving (the authenticated user). Any kitchen station (service class) can ask the waiter for the table number without needing to be told explicitly. When the waiter finishes and leaves (thread returns to pool), the notepad is cleared so the next table's details do not leak.

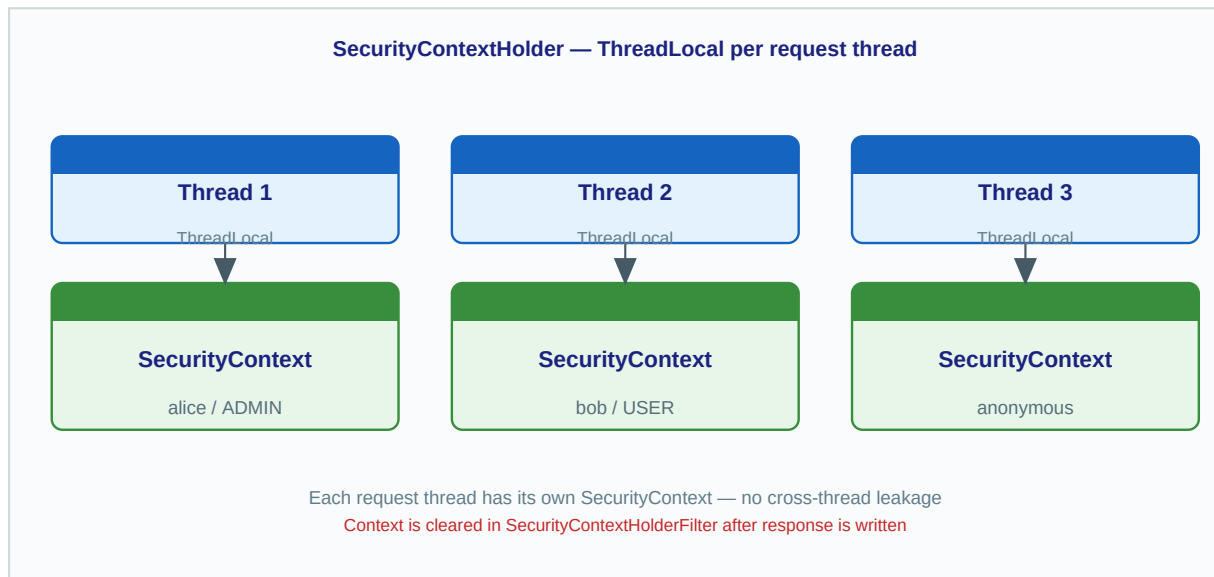
## 3. Minimal Code Example

```
// Reading the current user anywhere in the call stack
public String getCurrentUsername() {
    Authentication auth =
        SecurityContextHolder.getContext().getAuthentication();
    if (auth == null || !auth.isAuthenticated()) {
        return "anonymous";
    }
    return auth.getName();
}

// Setting context in a test
SecurityContext ctx = SecurityContextHolder.createEmptyContext();
ctx.setAuthentication(new UsernamePasswordAuthenticationToken(
    "testuser", null,
    List.of(new SimpleGrantedAuthority("ROLE_USER"))));
SecurityContextHolder.setContext(ctx);
```

## 4. Diagram





*Figure F — ThreadLocal ensures each thread sees only its own SecurityContext*

## 5. Behind the Scenes

**SecurityContextHolder** uses a strategy pattern. The default strategy is **MODE\_THREADLOCAL** which stores the context in a plain ThreadLocal. SecurityContextHolderFilter (order 500 in the filter chain) loads the context at request start from HttpSessionSecurityContextRepository and clears it after the response.

For **@Async methods**, Spring creates a new thread from a thread pool. That new thread has an empty ThreadLocal, so the SecurityContext is lost. Solutions:

- Set **MODE\_INHERITABLETHREADLOCAL**: SecurityContextHolder.setStrategyName(SecurityContextHolder.MODE\_INHERITABLETHREADLOCAL) — child threads inherit parent's context
- Use DelegatingSecurityContextAsyncTaskExecutor to wrap your Executor bean
- Use **@Async** with a custom TaskExecutor that propagates the SecurityContext

**SecurityContextHolderFilter** replaced SecurityContextPersistenceFilter in Spring Security 6. It still calls SecurityContextRepository.loadDeferredContext() at the start and saves at the end.

## 6. Realistic Practical Example

```
@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
        exec.setCorePoolSize(4);
        exec.setMaxPoolSize(16);
        exec.initialize();
        // Wrap so @Async methods receive SecurityContext from calling thread
        return new DelegatingSecurityContextAsyncTaskExecutor(exec);
    }
}

@Service
public class ReportService {
    @Async
    public CompletableFuture generateReport() {
        // SecurityContext is available here because of DelegatingSecurityContextExecutor
    }
}
```

```
String user = SecurityContextHolder.getContext()
    .getAuthentication().getName();
return CompletableFuture.completedFuture("Report for " + user);
}
}
```

## 7. Common Mistakes and Misconceptions

**[WARN]** Assuming `SecurityContext` is available in `@Async` methods by default — it is not

**[WARN]** Not clearing the context manually in unit tests — stale context bleeds into other tests

**[WARN]** Using `MODE_GLOBAL` in production — one shared context for all threads is a security disaster

**[OK]** Always wrap async executors with `DelegatingSecurityContextAsyncTaskExecutor`

**[OK]** In tests use `@WithMockUser` or set/clear the context explicitly in `@BeforeEach/@AfterEach`

## 1. Plain English Explanation

When you write **authorizeHttpRequests()** rules, each rule maps a URL pattern to an access decision. The rules are evaluated in declaration order — first match wins. The four common access expressions are:

- **permitAll()** — Anyone can access, including unauthenticated anonymous users
- **authenticated()** — Any logged-in user can access, regardless of roles
- **hasRole('ADMIN')** — User must have GrantedAuthority 'ROLE\_ADMIN' (prefix added automatically)
- **hasAuthority('reports:read')** — User must have exact GrantedAuthority 'reports:read' (no prefix)

## 2. Real-World Analogy

A shopping mall: the food court (**permitAll**) is open to everyone, including people just passing by. The changing rooms (**authenticated**) require you to have entered the mall (logged in). The staff room (**hasRole STAFF**) requires a staff badge. The safe room (**hasAuthority safe:open**) requires an exact named key — 'ROLE\_' prefix would be wrong, you need the exact key name.

## 3. Minimal Code Example

```
@Bean
public SecurityFilterChain chain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth
        // exact URL first, then patterns, then catch-all last
        .requestMatchers("/", "/login", "/register").permitAll()
        .requestMatchers("/public/**").permitAll()
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .requestMatchers("/reports/**").hasAuthority("reports:read")
        .requestMatchers("/api/**").authenticated()
        .anyRequest().authenticated()
    );
    return http.build();
}
```

## 4. Diagram

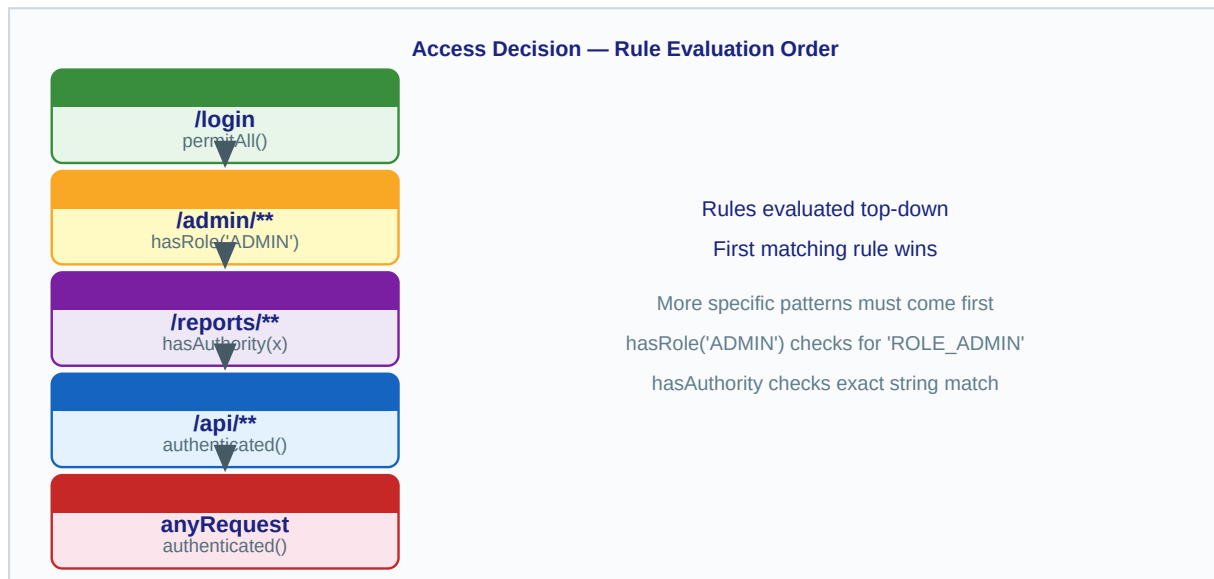


Figure G — Rules are evaluated in declaration order; first match is applied

## 5. Behind the Scenes

**AuthorizationFilter** calls `AuthorizationManager.check(supplier, request)` for each request. The `RequestMatcherDelegatingAuthorizationManager` iterates its ordered list of `RequestMatcherEntry` objects, finds the first whose `RequestMatcher` matches the URL, and delegates the access decision to that entry's `AuthorizationManager`.

**hasRole('ADMIN')** is syntactic sugar for `hasAuthority('ROLE_ADMIN')`. Spring automatically prepends 'ROLE\_' when you use `hasRole()`. Internally it creates an `AuthorityAuthorizationManager` which calls `Authentication.getAuthorities().contains(new SimpleGrantedAuthority('ROLE_ADMIN'))`.

**authenticated()** creates `AuthenticatedAuthorizationManager` which checks `!auth.isAnonymous()` — i.e., the `Authentication` is not `AnonymousAuthenticationToken`.

## 6. Realistic Practical Example

```
@Bean
public SecurityFilterChain chain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth
        .requestMatchers(HttpMethod.GET, "/products/**").permitAll()
        .requestMatchers(HttpMethod.POST, "/products/**").hasRole("SELLER")
        .requestMatchers("/admin/**")
            .access(new WebExpressionAuthorizationManager(
                "hasRole('ADMIN') and hasAuthority('admin:write')"))
        .requestMatchers("/actuator/health").permitAll()
        .requestMatchers("/actuator/**").hasRole("OPS")
        .anyRequest().authenticated()
    );
    return http.build();
}
```

## 7. Common Mistakes and Misconceptions

[WARN] Putting `anyRequest().permitAll()` before specific rules — it matches everything first

[WARN] Mixing `hasRole('ROLE_ADMIN')` with `hasRole()` — `hasRole` already adds the prefix, giving `ROLE_ROLE_ADMIN`

[WARN] Forgetting that `hasRole` and `hasAuthority` both check the **CURRENT** `SecurityContext`, not the DB

[OK] Always put most-specific matchers first, catch-all `anyRequest()` always last

[OK] Prefer hasAuthority for fine-grained permissions (read:order) and hasRole for broad groups

## 1. Plain English Explanation

JSON Web Token (JWT) authentication is stateless: the server stores nothing about the session. Instead, after login the server issues a signed token containing the user's identity and roles. On every subsequent request the client sends this token in the Authorization header. A filter validates the signature, extracts the claims, and sets the SecurityContext — all without any database lookup or session storage.

The token has three Base64url-encoded parts separated by dots: Header.Payload.Signature. The signature is produced with a secret key (HMAC-SHA256) or private key (RSA/ECDSA). The server verifies the signature on each request to ensure the token was not tampered with.

## 2. Real-World Analogy

A JWT is like a signed concert wristband. At the gate (login) staff stamp your wristband with an unforgeable mark (signature). At every area inside the venue (each API endpoint), security checks the wristband mark rather than calling the box office. The wristband carries your VIP level (roles) in text that anyone can read but only the venue can sign. An expired wristband (exp claim) is rejected at every checkpoint.

## 3. Minimal Code Example

```
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;
    private final UserDetailsService uds;

    @Override
    protected void doFilterInternal(HttpServletRequest req,
                                    HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtUtil.isValid(token)) {
                String username = jwtUtil.extractUsername(token);
                UserDetails ud = uds.loadUserByUsername(username);
                var auth = new UsernamePasswordAuthenticationToken(
                    ud, null, ud.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}
```

## 4. Diagram

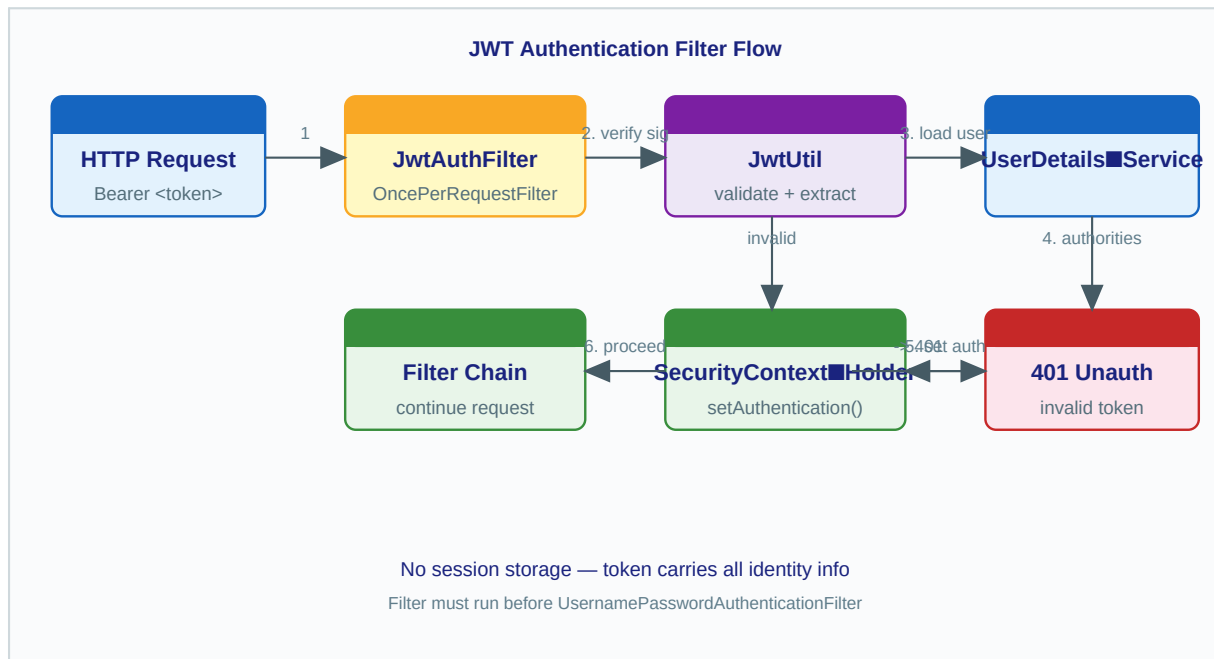


Figure H — JWT filter validates token and sets SecurityContext on every request

## 5. Behind the Scenes

The custom filter extends **OncePerRequestFilter** which guarantees `doFilterInternal()` is called exactly once per request, even in forward/include dispatch scenarios. It checks the Authorization header for a Bearer token.

`JwtUtil.isValid()` typically uses a library like `jjwt` or `nimbus-jose-jwt` to: parse the token, verify the HMAC-SHA256 signature using the stored secret key, check the 'exp' claim is not in the past, and check the 'iss' (issuer) and 'aud' (audience) claims.

After validation, **UsernamePasswordAuthenticationToken(principal, null, authorities)** is constructed. When this 3-argument constructor is used, `setAuthenticated(true)` is called automatically. This authenticated token is placed in `SecurityContextHolder`. No `AuthenticationManager` is involved — the filter bypasses the entire `ProviderManager` chain.

Register the filter with: `http.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)` so it runs before Spring's own auth filter.

## 6. Realistic Practical Example

```

@Component
public class JwtUtil {
    @Value("${jwt.secret}") private String secret;
    private static final long EXPIRY_MS = 86_400_000L; // 24h

    public String generate(UserDetails ud) {
        return Jwts.builder()
            .subject(ud.getUsername())
            .claim("roles", ud.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority).toList())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + EXPIRY_MS))
            .signWith(Keys.hmacShaKeyFor(secret.getBytes()))
            .compact();
    }

    public boolean isValid(String token) {

```

```
try {  
    Jwts.parser()  
        .verifyWith(Keys.hmacShaKeyFor(secret.getBytes()))  
        .build().parseSignedClaims(token);  
    return true;  
} catch (JwtException e) { return false; }  
}
```

## 7. Common Mistakes and Misconceptions

**[WARN] Storing sensitive data (passwords, PII) in JWT payload — it is Base64, not encrypted**

**[WARN] Using a weak or hardcoded secret key — minimum 256 bits for HMAC-SHA256**

**[WARN] Not checking token expiry — jjwt checks 'exp' automatically, but only if you parse correctly**

**[OK] Always use HTTPS — JWT tokens in headers are still visible without TLS**

**[OK] Keep JWT expiry short (15 min to 24h) and use refresh tokens for longer sessions**



## 1. Plain English Explanation

**Method security** moves access control from URL patterns into Java method calls. Annotate a service method with **@PreAuthorize** and Spring will check the expression before invoking the method. **@PostAuthorize** checks after the method returns and can inspect the return value. This is essential for domain-object level security where the decision depends on the data, not just the URL.

## 2. Real-World Analogy

A bank vault with two guards: the first guard (PreAuthorize) checks your clearance level before you enter the vault. Once you have retrieved the item, the second guard (PostAuthorize) inspects what you are carrying before you leave — for example, ensuring you can only take items that belong to your account. Both guards evaluate rules against the actual item, not just the door you used.

## 3. Minimal Code Example

```
@Service
public class AccountService {

    @PreAuthorize("hasRole('ADMIN') or #username == authentication.name")
    public Account getAccount(String username) {
        return accountRepo.findByUsername(username);
    }

    @PreAuthorize("hasAuthority('account:write') and #account.owner == authentication.name")
    public void updateAccount(Account account) {
        accountRepo.save(account);
    }

    @PostAuthorize("returnObject.owner == authentication.name or hasRole('ADMIN')")
    public Account findById(Long id) {
        return accountRepo.findById(id).orElseThrow();
    }
}
```

## 4. Diagram

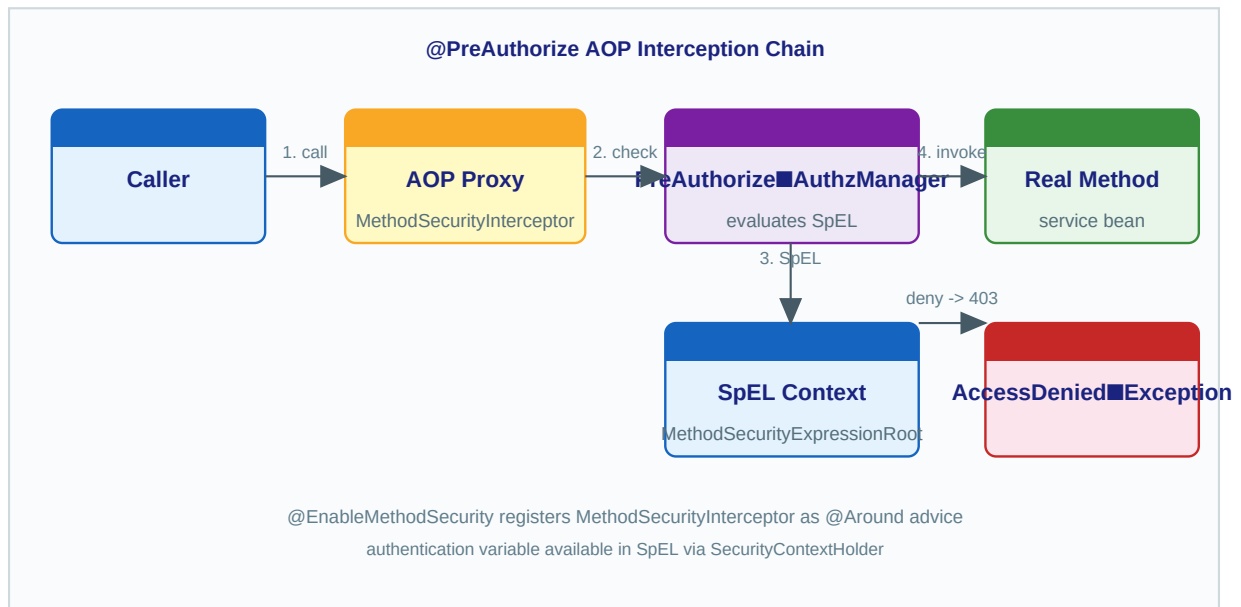


Figure 1 — AOP proxy intercepts the call; SpEL expression evaluated against SecurityContext

## 5. Behind the Scenes

**@EnableMethodSecurity** (or **@EnableGlobalMethodSecurity** in older Spring) imports **MethodSecurityConfiguration** which registers an AOP advisor: **AuthorizationManagerBeforeMethodInterceptor** for **@PreAuthorize** and **AuthorizationManagerAfterMethodInterceptor** for **@PostAuthorize**.

These interceptors implement **MethodInterceptor** (AOP Alliance). When a proxied method is called, the **@Around** advice fires first. For **@PreAuthorize** it calls **PreAuthorizeAuthorizationManager.check()** which parses the SpEL expression and evaluates it using **MethodSecurityExpressionRoot** as the root object.

**MethodSecurityExpressionRoot** exposes variables: *authentication* (current user), *returnObject* (for **@PostAuthorize**), and method parameters by name (e.g. *#username*) via reflection. Standard methods **hasRole()**, **hasAuthority()**, **isAuthenticated()** etc. are also available in SpEL through this root.

If the expression returns false, an **AccessDeniedException** is thrown, which **ExceptionHandler** catches and translates to a 403 response.

## 6. Realistic Practical Example

```

@EnableMethodSecurity
@Configuration
public class MethodSecConfig {}

@Service
public class DocumentService {

    // Only the document owner or an admin can delete
    @PreAuthorize("hasRole('ADMIN') or @documentRepo.findById(#id)"
        + ".orElseThrow().getOwner() == authentication.name")
    public void deleteDocument(Long id) {
        documentRepo.deleteById(id);
    }

    // Filter the returned list to only the caller's documents
    @PostFilter("filterObject.owner == authentication.name or hasRole('ADMIN')")
    public List findAll() {

```

```
        return documentRepo.findAll();  
    }  
}
```

## 7. Common Mistakes and Misconceptions

[WARN] Calling `@PreAuthorize`-protected methods from within the SAME bean — AOP proxy is bypassed

[WARN] Using `@EnableMethodSecurity` on the wrong config class — it must be picked up by the `ApplicationContext`

[WARN] Forgetting `proxyTargetClass=true` when the bean does not implement an interface

[OK] Always call `@PreAuthorize` methods through a Spring-injected reference, not 'this.'

[OK] `@PostFilter` is more efficient than filtering in code because Spring can short-circuit on denial

## 1. Plain English Explanation

**Cross-Site Request Forgery (CSRF)** is an attack where a malicious website tricks a user's browser into sending a state-changing request (POST, PUT, DELETE) to another site where the user is already authenticated. Because the browser automatically sends session cookies, the server cannot distinguish a legitimate request from a forged one.

Spring Security prevents CSRF with the **Synchroniser Token Pattern**: the server generates a secret random token per session and embeds it in every form. When a form is submitted, the server compares the submitted token to the stored one. A malicious site cannot read this token (same-origin policy) so forged requests fail the check.

## 2. Real-World Analogy

Imagine your bank's ATM gives you a one-time code printed on your receipt. To transfer money you must enter both your PIN (session cookie) AND the printed code (CSRF token). A thief who somehow forces you to walk to an ATM cannot complete the transfer because they do not have the code — only you received it in your hand.

## 3. Minimal Code Example

```
// CSRF is enabled by default – you need to include the token in forms
// Thymeleaf does this automatically with th:action

// In plain HTML:
// <input type="hidden" name="${_csrf.parameterName}" value="${_csrf.token}"/>

// In a REST controller test:
mockMvc.perform(post("/transfer")
    .with(csrf()) // MockMvc CSRF support
    .param("amount", "100")
    .param("to", "bob"))
    .andExpect(status().isOk());

// Disable CSRF only for stateless REST APIs that use JWT (no session cookies):
http.csrf(csrf -> csrf.disable());
```

## 4. Diagram

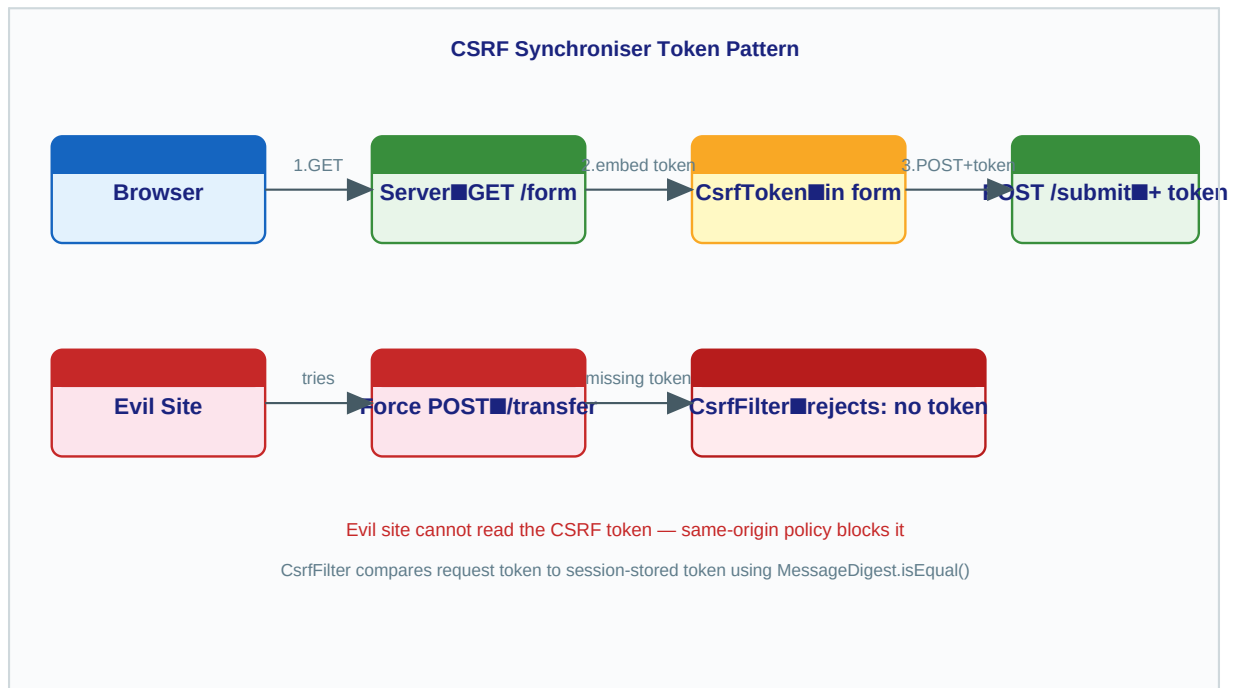


Figure J — Legitimate requests include the token; forged requests cannot obtain it

## 5. Behind the Scenes

**CsrfFilter** (order 700 in the filter chain) is the guard. On GET requests it loads or generates a **CsrfToken** from **CsrfTokenRepository** (default: `HttpSessionCsrfTokenRepository`, which stores the token in the HTTP session).

On state-changing requests (POST, PUT, PATCH, DELETE) it extracts the submitted token from the request header (X-CSRF-TOKEN) or parameter (`_csrf`) and compares it to the stored token using **MessageDigest.isEqual()** for constant-time comparison. A mismatch throws **InvalidCsrfTokenException**.

For REST APIs that use JWT and *no session cookies*, CSRF is not needed and should be disabled. CSRF only matters when the browser automatically sends credentials (cookies). JWT in an Authorization header is not automatically sent by the browser to other domains.

**CookieCsrfTokenRepository.withHttpOnlyFalse()** is used for SPAs (React, Angular): the token is stored in a cookie (readable by JavaScript) and sent back in a request header rather than a form parameter.

## 6. Realistic Practical Example

```

// SPA (React) configuration – cookie-based CSRF token
@Bean
public SecurityFilterChain chain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf
            .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
            .csrfTokenRequestHandler(new SpaCsrfTokenRequestHandler()))
        .authorizeHttpRequests(a -> a
            .requestMatchers(HttpMethod.GET, "/*").permitAll()
            .anyRequest().authenticated());
    return http.build();
}

// React client reads the XSRF-TOKEN cookie and sends it as a header:
  
```

```
// headers: { 'X-XSRF-TOKEN': getCookieValue('XSRF-TOKEN') }
```

## 7. Common Mistakes and Misconceptions

[WARN] Disabling CSRF for MVC applications that use session cookies and form logins

[WARN] Only checking CSRF on POST — PUT, PATCH, and DELETE are equally vulnerable

[WARN] Using httpOnly=true for the CSRF cookie in a SPA — JavaScript cannot read it to send back

[OK] Disable CSRF only for stateless APIs that never use session cookies

[OK] For SPAs use `CookieCsrfTokenRepository.withHttpOnlyFalse()` and send via X-XSRF-TOKEN header

## Spring Security Quick-Reference Cheat Sheet

| Concept                 | Key Class / Interface                                     | Purpose                     | Common Pitfall                         |
|-------------------------|-----------------------------------------------------------|-----------------------------|----------------------------------------|
| Authentication vs Authz | Authentication, AuthorizationFilter                       | Who + What can you do?      | Confusing 401 and 403                  |
| SecurityFilterChain     | FilterChainProxy, DelegatingFilterProxy                   | Ordered filter list per URL | Missing @Order on multiple chains      |
| Auth Flow               | UsernamePasswordAuthenticationFilter, ProviderManager     | POST /login processing      | Not throwing UsernameNotFoundException |
| UserDetailsService      | UserDetailsService, UserDetails                           | Load user from DB           | FetchType.LAZY on roles                |
| PasswordEncoder         | BCryptPasswordEncoder, DelegatingPasswordEncoder          | Hash and verify passwords   | Cost factor too low in prod            |
| SecurityContextHolder   | SecurityContextHolder, ThreadLocal                        | Per-thread auth storage     | Lost context in @Async                 |
| HTTP Security Config    | AuthorizationFilter, RequestMatcherDelegating             | URL access rules            | hasRole adds ROLE_ prefix              |
| JWT Auth                | OncePerRequestFilter, UsernamePasswordAuthenticationToken | Stateless token validation  | Storing sensitive data in payload      |
| Method Security         | PreAuthorizeAuthorizationManager, AOP Proxy               | SpEL-based method guards    | Calling protected method via 'this'    |
| CSRF                    | CsrfFilter, CsrfTokenRepository                           | Forged request prevention   | Disabling CSRF for session apps        |

## Filter Chain Execution Order Reference

| Order | Filter                               | Responsibility                                 |
|-------|--------------------------------------|------------------------------------------------|
| 100   | DisableEncodeUrlFilter               | Prevent session ID in URLs                     |
| 500   | SecurityContextHolderFilter          | Load/save SecurityContext from session         |
| 700   | CsrfFilter                           | Validate CSRF token on state-changing requests |
| 800   | LogoutFilter                         | Handle POST /logout                            |
| 1100  | UsernamePasswordAuthenticationFilter | Process form login POST /login                 |
| 1500  | DefaultLoginPageGeneratingFilter     | Render built-in login page                     |

|      |                            |                                            |
|------|----------------------------|--------------------------------------------|
| 3000 | ExceptionTranslationFilter | Translate AccessDeniedException to 401/403 |
| 3100 | AuthorizationFilter        | Enforce URL access rules (last filter)     |